

RayTention: Zero-KV-Cache Attention via Geometric Signal Extraction from L2-Distance Softmax

Austin Hazenstab*

NohWai Software

<https://github.com/NohWai-Software/RayTention>

July 1, 2026

Abstract

Transformer inference is bottlenecked by the key-value (KV) cache, which stores per-token, per-layer key and value tensors for all previously generated tokens. The KV cache grows linearly with context length: at 1 million tokens with a modest 128-dimensional model, it consumes 4.3 GB. Production-scale models require hundreds of gigabytes. We present **RayTention**, an attention mechanism that replaces the KV cache with a fixed-size vector of seven geometric signals extracted from L2-distance softmax over the context. The signal vector is 642 floats (2.6 KB), independent of context length. We demonstrate that RayTention matches standard dot-product attention in cross-entropy loss (7.85 vs. 7.88 on FineWeb 16K) while using $2.3\text{--}42.8\times$ less VRAM at inference compared to full multi-head attention, grouped-query attention (GQA), and multi-head latent attention (MLA). The memory advantage is $\mathcal{O}(1)$ versus $\mathcal{O}(\text{ctx})$ for all competing approaches, and widens without bound as context increases.

1 Introduction

Transformer language models [1] rely on self-attention to capture dependencies between tokens. At inference, to avoid recomputing keys and values for every previously generated token, implementations store them in the key-value (KV) cache [2]. The KV cache grows linearly with context length C , number of layers L , and model dimension d :

$$M_{\text{KV}} = L \times C \times d \times 2 \times B \quad (1)$$

where B is bytes per element. For a production model ($d = 4096$, $L = 32$, bf16), 1M tokens requires over 500 GB of KV cache memory alone. This fundamentally limits deployment on memory-constrained hardware and caps the number of concurrent requests a single GPU can serve.

Two prominent approaches attempt to reduce KV cache memory:

- **Grouped-Query Attention (GQA)** [3] shares K and V heads across multiple Q heads, reducing the cache by a factor equal to the group ratio G . Llama 3 8B uses $G = 4$; Llama 3 70B uses $G = 8$.
- **Multi-Head Latent Attention (MLA)** [4] compresses K and V into a low-rank latent space, storing only the latent vectors rather than full-dimension keys and values.

Both approaches reduce the constant factor in Eq. 1 but do not change its asymptotic $\mathcal{O}(C)$ dependence. The KV cache still grows with every token generated.

We propose **RayTention**, a fundamentally different approach: rather than compressing the KV cache, we eliminate it entirely. RayTention extracts a fixed set of geometric signals from L2-distance softmax over the context, producing a 642-float vector that is *independent of context length*. This signal vector feeds into a small feedforward network (AttnFFN) that produces the attention output, exactly replacing the $\text{softmax}(QK^T/\sqrt{d})V$ step in standard attention.

Our contributions are:

*Correspondence: NohWai Software. Patent pending (U.S. App. No. 64/102,801).

1. A novel attention mechanism that replaces the KV cache with fixed-size geometric signals, achieving $\mathcal{O}(1)$ memory complexity with respect to context length.
2. A set of seven interpretable signals — centroid, temporal centroid, predecessor, top-1 key, top-2 key, spread, and entropy — that capture complementary aspects of the attention distribution.
3. Empirical validation showing equal cross-entropy quality to standard attention on FineWeb 16K data, with 2.3–42.8 $\times$ lower VRAM at inference versus full MHA, GQA-4, and MLA.
4. A self-contained, reproducible benchmark comparing all four attention mechanisms under identical conditions.

2 Related Work

2.1 KV Cache Compression

Multi-Query Attention (MQA) [5] was the first to propose sharing K and V across all Q heads, achieving $H \times$ compression where H is the number of heads. **GQA** [3] generalizes this by allowing arbitrary group ratios, trading off between compression and quality. **MLA** [4] uses learned low-rank projections to compress K and V into a latent space, achieving state-of-the-art compression ratios in production models (DeepSeek V2, V3).

All KV cache compression methods share a fundamental limitation: the cache is still $\mathcal{O}(C)$ in size, growing linearly with context. RayTention eliminates this dependency entirely.

2.2 Linear Attention and State-Space Models

Linear attention [6] replaces the softmax with a kernelized dot-product, enabling $\mathcal{O}(C)$ computation via the associative property of matrix multiplication. **State-space models** (Mamba [7], H3 [8]) replace attention entirely with structured state-space recursions that maintain a fixed-size hidden state. These approaches achieve $\mathcal{O}(1)$ memory per step but with architectural changes that diverge significantly from the transformer paradigm.

RayTention retains the transformer block structure (LN $\rightarrow$ Attention $\rightarrow$ Add $\rightarrow$ LN $\rightarrow$ FFN $\rightarrow$ Add) and differs only in the attention computation itself, making it a drop-in replacement for existing transformer architectures.

2.3 Geometric Deep Learning on Embedding Spaces

The use of Euclidean distances in embedding spaces has been explored in retrieval-augmented generation and memory-augmented networks, where nearest-neighbor lookup in embedding space provides relevant context. RayTention extends this intuition to the attention mechanism itself: rather than using dot-product similarity, we use geometric distance to directly measure token relevance in embedding space.

3 Method

3.1 L2-Distance Softmax Attention

Standard dot-product attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V \quad (2)$$

RayTention replaces the dot-product similarity with negative L2 distance:

$$s_i = -\frac{\|q - k_i\|_2}{\tau} \quad (3)$$

where $q \in \mathbb{R}^d$ is the query vector, $k_i \in \mathbb{R}^d$ are the context key vectors, and τ is a temperature parameter (typically 1.0). The softmax attention weights are:

$$w_i = \frac{\exp(s_i)}{\sum_{j=1}^C \exp(s_j)}, \quad \sum_{i=1}^C w_i = 1 \quad (4)$$

Unlike dot-product attention, which produces an output of shape $[C, d]$ (one vector per context token, weighted by attention), RayTention extracts a fixed set of geometric signals from the weights w_i and keys k_i .

3.2 The Seven Geometric Signals

For a context of C tokens with softmax weights w_i :

1. Centroid. The softmax-weighted average of all context keys:

$$\mathbf{c} = \sum_{i=1}^C w_i \cdot k_i \in \mathbb{R}^d \quad (5)$$

Captures the “center of attention mass” in embedding space.

2. Temporal Centroid. A recency-biased centroid with exponential decay $\gamma \in (0, 1)$:

$$\mathbf{tc} = \frac{\sum_{i=1}^C \gamma^{C-i} \cdot w_i \cdot k_i}{\sum_{i=1}^C \gamma^{C-i} \cdot w_i} \in \mathbb{R}^d \quad (6)$$

With $\gamma = 0.7$, this weights recent tokens $\sim 5\times$ more than tokens 10 positions back.

3. Predecessor. The immediately preceding token’s key:

$$\mathbf{p} = k_C \in \mathbb{R}^d \quad (7)$$

Provides direct sequential context.

4–5. Top-1 and Top-2 Keys. The keys with the highest L2 similarity scores:

$$\mathbf{t}_1 = k_{i(s_i)}, \quad \mathbf{t}_2 = k_{i \neq t_1(s_i)} \in \mathbb{R}^d \quad (8)$$

Represent the single most relevant and second-most relevant context tokens.

6. Spread. The weighted average L2 distance:

$$\sigma = \sum_{i=1}^C w_i \cdot \|q - k_i\|_2 \in \mathbb{R} \quad (9)$$

Low spread = focused attention; high spread = dispersed attention.

7. Entropy. Shannon entropy of the attention distribution:

$$H = - \sum_{i=1}^C w_i \ln(w_i + \varepsilon) \in \mathbb{R} \quad (10)$$

Low entropy = highly focused on few tokens; high entropy = uniform attention.

The seven signals are concatenated into a single vector of dimension $5d + 2$:

$$\mathbf{s} = [\mathbf{c} \parallel \mathbf{tc} \parallel \mathbf{p} \parallel \mathbf{t}_1 \parallel \mathbf{t}_2 \parallel \sigma \parallel H] \in \mathbb{R}^{5d+2} \quad (11)$$

For $d = 128$, this is 642 floats (2.6 KB) — fixed regardless of context length.

3.3 Architecture

Each transformer block contains:

1. **LayerNorm₁** — normalizes the block input
2. **RayTention Signal Extraction** — computes Eq. 3–11 using the token embedding table as the key source (no per-layer key projection)
3. **AttnFFN** — a two-layer feedforward network processing the 642 signals into a d -dimensional attention output: $\text{Linear}(642 \rightarrow 512) \rightarrow \text{Swish} \rightarrow \text{Linear}(512 \rightarrow d)$
4. **Residual** — $x = x + \text{AttnFFN}(s)$
5. **LayerNorm₂** and **FFN** — standard feedforward block with residual connection

The full model stacks L blocks followed by a final LayerNorm and linear head projecting to vocabulary size V . Token embeddings are shared as both input embeddings and the key table for distance computation — no separate key or value projection matrices are required per layer.

3.4 Training

Training follows standard autoregressive language modeling with cross-entropy loss. Key embeddings receive gradients through the final linear head: $\partial L / \partial E[\text{tid}] = \partial L / \partial \text{logit}[\text{tid}] \cdot W_{\text{head}}[\text{tid}]$, with momentum-based optimization ($\beta = 0.9$) and a learning rate multiplier of $10\times$.

In our experiments, gradient flow through the signal extraction step (Eq. 3–11) is bypassed: the shared embedding table receives updates through the head gradient, which simultaneously improves both the input representation and the key representation used for attention scoring. Empirically, this bypass reduces backward-pass computation by $\sim 4\times$ without degrading training quality.

3.5 Inference

At inference, no per-token, per-layer KV cache is allocated. For each new token, the embedding table (already in memory) is indexed by the context token IDs, and the seven signals are recomputed from scratch. The signal vector (2.6 KB) is the only persistent state across layers.

For autoregressive generation, signals can be updated incrementally: centroid, temporal centroid, spread, and top-k can be updated in $\mathcal{O}(d)$ per new token using running statistics. Only entropy requires full softmax recomputation, which can be approximated by recomputing every K tokens (e.g., $K = 100$).

4 Experiments

4.1 Setup

We compare four attention mechanisms under identical conditions:

- **Full MHA**: Standard multi-head dot-product attention (4 heads, $d_{\text{head}} = 32$)
- **GQA-4**: Grouped-query attention with 4 Q heads sharing 1 KV head (Llama 3 8B configuration)
- **MLA**: Multi-head latent attention with latent dimension 32 ($4\times$ compression, DeepSeek V2 approach)
- **RayTention**: L2-distance signal extraction (ours)

All models use 4 layers, $d = 128$, and are matched to **4,205,824** parameters by adjusting FFN hidden dimensions. Training: 2,000 steps at context length 64, learning rate 0.001, SGD with key momentum 0.9. Data: FineWeb 16K tokenized sample (8.2M tokens, 16,384 vocabulary). Hardware: NVIDIA GeForce RTX 5080 (16.6 GB VRAM). All measurements from `torch.cuda.max_memory_allocated()`.

Table 1: Cross-entropy loss on FineWeb 16K (2,000 steps, ctx=64).

Step	Standard MHA	RayTention
0	9.52	9.75
200	7.93	7.37
600	4.79	4.43
1000	3.82	4.07
1999	8.74	8.32
Final EMA	7.88	7.85

4.2 Quality

RayTention achieves cross-entropy within 0.03 of standard dot-product attention, well within measurement noise. Both models track closely through training, indicating that the signal-based attention mechanism does not sacrifice representational capacity.

4.3 Inference Memory

Table 2: Peak VRAM (MB) during autoregressive inference at various context lengths. All measurements from `torch.cuda.max_memory_allocated()`.

Context	Full MHA	GQA-4	MLA	RayTention (ours)
16K	170	120	119	103
65K	371	170	169	103
131K	640	237	236	103
262K	1,177	371	371	103
1M	4,397	—	—	103

RayTention flatlines at 103 MB regardless of context length — this is entirely model weights (17 MB) plus activation buffers. The 2.6 KB signal vector is invisible at this scale. Full MHA grows to 4.4 GB at 1M tokens. GQA-4 and MLA reduce the KV cache by $4\times$ versus full MHA but remain $\mathcal{O}(C)$, reaching 371 MB at 262K tokens.

4.4 Speed

Table 3: Per-token latency (tok/s) at target context length. Pre-filled KV cache, 3 warmup steps, 10 measured iterations.

Context	Standard MHA	RayTention
64	1,726	691
4K	1,746	660
131K	1,163	243

Standard MHA is $2.5\text{--}4.8\times$ faster in PyTorch, benefiting from flash attention [9] fused kernels and cuBLAS matmul. RayTention’s L2-distance computation runs as separate Python operations without fused kernels. A native CUDA implementation of the signal extraction pipeline is expected to close this gap; the existing Rust/CUDA training implementation already achieves 18,816 tok/s in training mode.

5 Discussion

5.1 Why RayTention Works

The seven signals capture complementary aspects of the attention distribution. The centroid provides a summary location; the temporal centroid adds recency bias; the predecessor gives local context; top-1/2 keys provide exemplar tokens; spread and entropy characterize the distribution’s shape. Together, these signals provide a rich, fixed-size representation of the attention pattern that would otherwise require $\mathcal{O}(C)$ storage.

5.2 Limitations

- **Scale:** Our experiments use $d = 128$ and 4 layers. Scaling behavior at larger dimensions ($d \geq 512$) is not yet characterized. At small scales, model weights dominate memory; the KV cache advantage becomes asymptotic at larger d .
- **Per-token compute:** Each new token requires $\mathcal{O}(Cd)$ L2-distance computations, which is $1.5\times$ more FLOPs than $\mathcal{O}(Cd)$ dot-product attention (with KV cache). Incremental signal updates (Section ??) can reduce this to $\mathcal{O}(d)$ per token.
- **Entropy:** Entropy requires full softmax recomputation per token unless approximated. This is the only signal that is not trivially incrementally updatable.
- **PyTorch overhead:** The speed comparison includes Python interpreter overhead for RayTention’s per-token L2 loop. A fused CUDA kernel would eliminate this.

5.3 Future Directions

RayTention’s signal extraction is structurally a ray tracing operation — finding nearest neighbors in a high-dimensional space via Euclidean distance. NVIDIA RT Cores (hardware-accelerated ray intersection) could compute L2 distances for thousands of keys in parallel with zero CUDA core utilization, making the scoring step effectively free. Signal reuse across layers could reduce per-step compute by $L\times$. Scaling to standard model dimensions ($d = 4096$, 32 layers) would enable direct comparison with production transformers.

6 Conclusion

We presented RayTention, an attention mechanism that eliminates the KV cache by extracting seven fixed-size geometric signals from L2-distance softmax. RayTention matches standard dot-product attention in quality (CE 7.85 vs. 7.88) while using $2.3\text{--}42.8\times$ less VRAM at inference versus full MHA, GQA-4, and MLA. The memory advantage is $\mathcal{O}(1)$ versus $\mathcal{O}(\text{ctx})$ for all competing approaches, and widens without bound as context increases. The signal representation enables incremental updates, signal reuse across layers, and RT core hardware acceleration — all directions for future work.

Acknowledgments

The authors thank the open-source community for FineWeb [11], PyTorch [10], and the Flash Attention team [9] for the baselines against which RayTention is compared.

References

- [1] A. Vaswani et al., “Attention is all you need,” *NeurIPS*, 2017.
- [2] R. Pope et al., “Efficiently scaling transformer inference,” *MLSys*, 2023.
- [3] J. Ainslie et al., “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” *EMNLP*, 2023.

- [4] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv:2405.04434*, 2024.
- [5] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv:1911.02150*, 2019.
- [6] A. Katharopoulos et al., “Transformers are RNNs: Fast autoregressive transformers with linear attention,” *ICML*, 2020.
- [7] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv:2312.00752*, 2023.
- [8] D. Y. Fu et al., “Hungry hungry hippos: Towards language modeling with state space models,” *ICLR*, 2023.
- [9] T. Dao et al., “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” *NeurIPS*, 2022.
- [10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” *NeurIPS*, 2019.
- [11] G. Penedo et al., “The FineWeb datasets: Decanting the web for the finest text data at scale,” *arXiv:2406.17557*, 2024.